

성능테스트를 위한 Jenkins (완)

1. Jenkins 컨테이너를 삭제해도 기존 파이프라인을 유지할 수 있도록 volume 을 EC2에 만들기

```
docker volume create jenkins_test_home
```

```
# 현재 볼륨 보기
```

```
docker volume ls
```

2. 성능테스트를위한 Jenkins 컨테이너 실행

```
docker run -d --name jenkins-test \  
  --network my-network \  
  -p 9090:8080 \  
  -v jenkins_test_home:/var/jenkins_home \  
  jenkins/jenkins:its
```

```
# json 으로 보내기 위해 jq 설치
```

```
apt update && apt install -y jq
```

3. Jenkins 컨테이너에 접속

```
docker exec -it jenkins-test bash
```

4. JMeter를 실행하려면 자바가 필요, JDK 설치

```
apt update && apt install -y openjdk-17-jdk
```

```
# 버전 확인
```

```
java -version
```

5. JMeter 설치

```
# JMeter 설치를 위한 wget 설치
apt update && apt install -y wget
# 버전 확인
wget --version

# JMeter 설치
wget https://archive.apache.org/dist/jmeter/binaries/apache-jmeter-5.6.2.tgz
tar -xvzf apache-jmeter-5.6.2.tgz
mv apache-jmeter-5.6.2 /opt/jmeter
echo 'export PATH="/opt/jmeter/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# 버전 확인
jmeter -v
```

6. Lighthouse 설치

```
apt install -y nodejs npm
npm install -g lighthouse

# 버전 확인
lighthouse --version

# lighthouse 사용을 위해 chrome 설치
apt-get update && apt-get install -y chromium

# frontend 프로젝트 실행을 위해 nodejs, yarn 설치
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs
npm install -g yarn@1.22.22
```

7. 도메인:9090 으로 Jenkins 웹페이지 접속

```
http://i12b108.p.ssafy.io:9090/

# EC2에서 docker logs jenkins-test 로 비밀번호를 찾고
```

http://i12b108.p.ssafy.io:9090/ 여기에 나오는 곳에 입력

왼쪽에 있는 install~ 누르고, 계정 생성, 그리고 url 설정 하면 Jenkins 웹 페이지 접속

8. pipeline 설정

새로운 item → Pipeline 선택 후, item 이름 입력 후 ok 버튼으로 item 생성

현재 파이프라인

```
pipeline {
  agent any
  parameters {
    string(name: 'gitUrl', defaultValue: 'https://5HVt9H2Zws6AT_FQdxYy5HV
    string(name: 'frontendPath', defaultValue: 'frontend', description: 'Fronter
    string(name: 'branch', defaultValue: 'develop', description: 'Git Branch')
    string(name: 'repold', defaultValue: '761731', description: 'Repository ID')
    string(name: 'build', defaultValue: 'yarn', description: 'Build tool (npm/yar
  }
  environment {
    TEST_TYPE = " //  전역 환경 변수 초기화
  }
  stages {
    stage('Cleanup') {
      steps {
        deleteDir()
      }
    }
    stage('Clone Repository') {
      steps {
        checkout([
          $class: 'GitSCM',
          branches: [[name: "${params.branch}"]],
          userRemoteConfigs: [[url: "${params.gitUrl}"]],
          extensions: [[$class: 'CloneOption', shallow: true, depth: 1, noTags
        ])
      }
    }
  }
}
```

```

}
stage('Install Dependencies') {
  steps {
    dir("${params.frontendPath}") {
      script {
        if (!fileExists('node_modules')) {
          echo "📦 Installing dependencies using ${params.build}..."
          if (params.build == "npm") {
            sh '''
              npm cache clean --force
              npm install --legacy-peer-deps || (echo "⚠️ npm install fail
            } else if (params.build == "yarn") {
              sh 'yarn install --silent'
            } else {
              error "❌ Invalid build tool. Must be 'npm' or 'yarn'."
            }
          } else {
            echo "✅ node_modules already exists. Skipping installation."
          }
        }
      }
    }
  }
}

stage('Start React Server') {
  steps {
    dir("${params.frontendPath}") {
      script {
        echo "🚀 Starting React server using ${params.build}..."
        if (params.build == "npm") {
          sh '''
            nohup npm run dev -- --host 0.0.0.0 --port 3000 > react_ser
          '''
        } else if (params.build == "yarn") {
          sh '''
            nohup yarn dev --host 0.0.0.0 --port 3000 > react_server.log
          '''
        } else {
          error "❌ Invalid build tool. Must be 'npm' or 'yarn'."
        }
      }
    }
  }
}

```

```

    }

    // 서버가 뜰 때까지 대기
    sh '''
    for i in {1..30}; do
        if curl -s http://localhost:3000 | grep -q "<title>"; then
            echo "✅ React 서버 정상 실행됨."
            break
        fi
        sleep 1
    done
    '''
}
}
}
}

stage('Run Lighthouse Test') {
    steps {
        dir("${params.frontendPath}") {
            script {
                echo "🚦 Running Lighthouse Test..."
                def lighthouseStatus = sh(
                    script: '''
                    lighthouse http://localhost:3000 \
                        --output=json --output-path=lighthouse-report.json \
                        --chrome-flags="--no-sandbox --headless --disable-gpu" \
                        --preset=desktop \
                    || echo "❌ Lighthouse test failed!"
                    ''',
                    returnStatus: true
                )

                if (lighthouseStatus != 0) {
                    echo "❌ Lighthouse execution failed. Marking build as FAILURE"
                    currentBuild.result = 'FAILURE'
                    error("Lighthouse execution failed")
                }
            }
        }
    }
}

```

```

echo "✅ Lighthouse test completed successfully."

// JSON에서 값 추출 및 검증
def lighthouseMetrics = sh(
    script: '''
        cat lighthouse-report.json | jq '{
            "performance": .categories.performance.score,
            "fcp": .audits["first-contentful-paint"].numericValue,
            "lcp": .audits["largest-contentful-paint"].numericValue,
            "tbt": .audits["total-blocking-time"].numericValue,
            "cls": .audits["cumulative-layout-shift"].numericValue,
            "si": .audits["speed-index"].numericValue
        }'
    '''
    ,
    returnStdout: true
).trim()

echo "Lighthouse Metrics: ${lighthouseMetrics}"

def performance = sh(script: "echo '${lighthouseMetrics}' | jq -r '.performance'"
def fcp = sh(script: "echo '${lighthouseMetrics}' | jq -r '.fcp'"
def lcp = sh(script: "echo '${lighthouseMetrics}' | jq -r '.lcp'"
def tbt = sh(script: "echo '${lighthouseMetrics}' | jq -r '.tbt'"
def cls = sh(script: "echo '${lighthouseMetrics}' | jq -r '.cls'"
def si = sh(script: "echo '${lighthouseMetrics}' | jq -r '.si'"

// 만약 주요 Lighthouse 값이 0이면 실패로 처리
if (performance == "0" && fcp == "0" && lcp == "0" && tbt == "0" && cls == "0" && si == "0") {
    echo "❌ Lighthouse test detected invalid performance value"
    currentBuild.result = 'FAILURE'
    error("Lighthouse execution produced invalid results")
}
}
}
}
}
}
stage('Send Lighthouse Report to spring-report') {
    steps {

```

```

dir("${params.frontendPath}") {
  sh '''
  echo "🚀 Sending Lighthouse Report as JSON to spring-test..."
  cat lighthouse-report.json | jq '{
    "performance": .categories.performance.score,
    "accessibility": .categories.accessibility.score,
    "bestPractices": .categories["best-practices"].score,
    "seo": .categories.seo.score,
    "fcp": .audits["first-contentful-paint"].numericValue,
    "si": .audits["speed-index"].numericValue,
    "lcp": .audits["largest-contentful-paint"].numericValue,
    "cls": .audits["cumulative-layout-shift"].numericValue,
    "tbt": .audits["total-blocking-time"].numericValue
  }' > metrics.json

  curl -X POST "http://spring-report:8080/api/report/lighthouse-res
    -H "Content-Type: application/json" \
    --data @metrics.json || echo "❌ Failed to send Lighthouse me

# repold를 헤더로 추가하여 요청 전송
curl -X POST "http://spring-report:8080/api/report/lighthouse-res
  -H "Content-Type: application/json" \
  -H "repold: $repold" \
  -H "branch: $branch" \
  --data @metrics.json || echo "❌ Failed to send Lighthouse me
'''
}
}
}
stage('Stop React Server') {
  steps {
    dir("${params.frontendPath}") {
      sh '''
      echo "🛑 Stopping React server..."
      kill -9 $(cat react_server.pid) || echo "⚠️ React server already stop
      '''
    }
  }
}

```

```
}  
}  
}
```